

Lezione 3: Sistemi Operativi Strutture



Obiettivi

- Definire i servizi forniti da un Sistema Operativo
- Introdurre le **chiamate di sistema** che forniscono i servizi del sistema
- Discutere la strutturazione di un Sistema Operativo

Servizi degli SO

- I sistemi operativi forniscono un ambiente per l'esecuzione di programmi e servizi a **programmi** e **utenti**



Servizi di Sistemi Operativi

- I sistemi operativi forniscono un ambiente per l'esecuzione di programmi e servizi a programmi e utenti
 - **Interfaccia Utente** – quasi tutti i sistemi le forniscono (**UI**).
 - ▶ Diversi tipi **Command-Line (CLI)**, **Graphics User Interface (GUI)**, **Batch**
 - **Esecuzione di Programmi** - Il sistema deve essere in grado di caricare un programma in memoria ed eseguire quel programma, terminare l'esecuzione, normalmente o in modo anomalo
 - **Operazioni I/O** - Un programma in esecuzione può richiedere I/O, che può coinvolgere un file o un dispositivo I/O.
 - ▶ Il SO deve nascondere la complessità del dispositivo offrendo agli utenti un interfacciamento semplice ed uniforme

Servizi degli SO

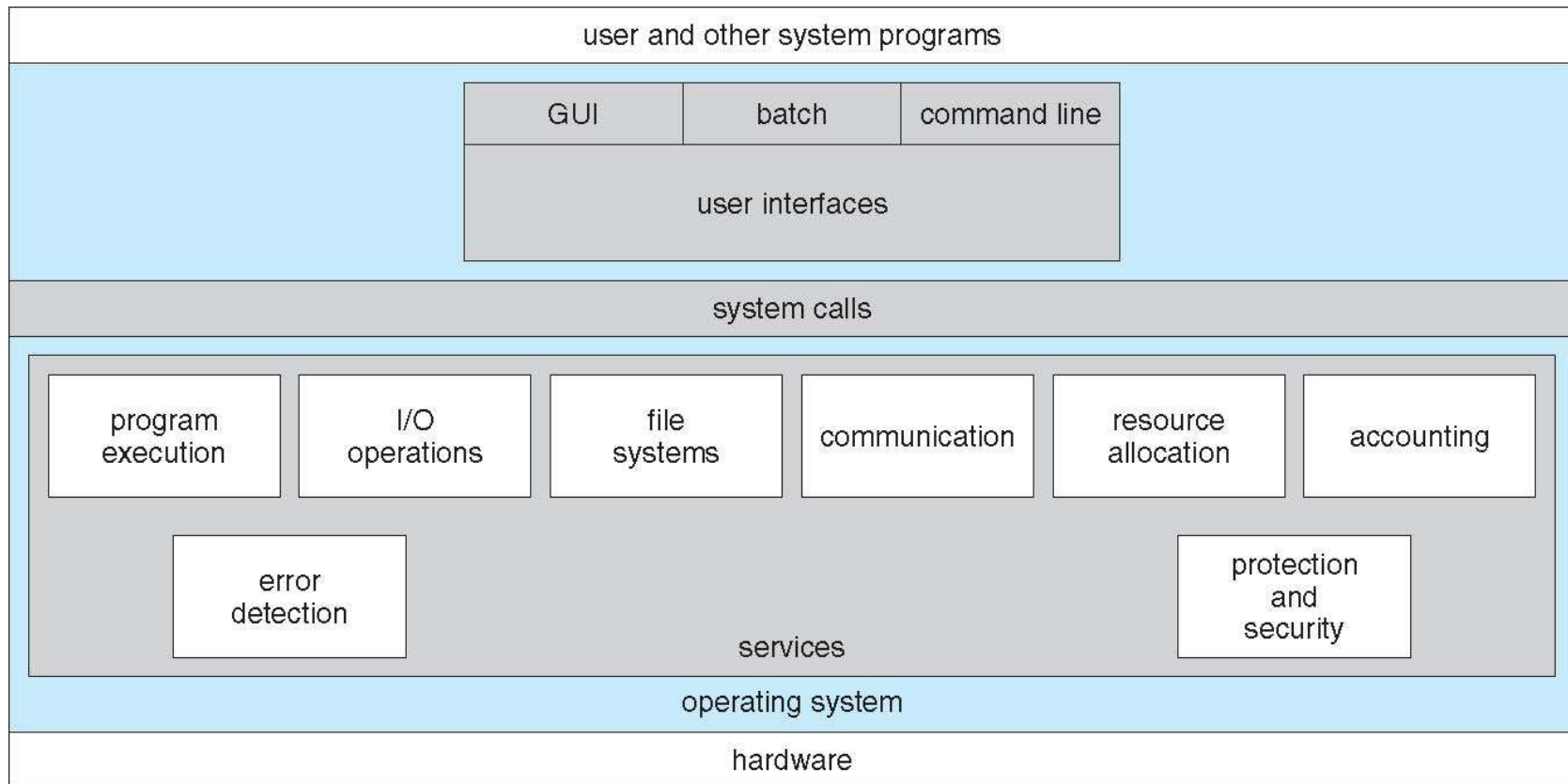
- Un insieme di servizi del sistema operativo:
 - **Gestione File-system** - I programmi devono leggere e scrivere file e directory, crearli ed eliminarli, ricercarli, elencare le informazioni sui file, gestire i permessi. Accesso frequenti richiede gestione oculata.
 - **Comunicazione** – I processi possono scambiare informazioni, sullo stesso computer o tra computer in una rete
 - **Rilevamento di Errori** – Il sistema operativo deve essere costantemente consapevole di possibili errori
 - ▶ Può verificarsi nella CPU e nell'hardware di memoria, nei dispositivi I/O, nel programma utente
 - ▶ Per ogni tipo di errore, il sistema operativo dovrebbe intraprendere l'azione appropriata per garantire un'elaborazione corretta e coerente
 - ▶ Le funzionalità di debug possono migliorare notevolmente le capacità dell'utente e del programmatore di utilizzare in modo efficiente il sistema

Servizi degli SO

- Insieme di funzioni del sistema operativo per garantire il funzionamento efficiente del sistema stesso tramite la condivisione delle risorse
- **Allocazione delle risorse** - quando più utenti o più lavori vengono eseguiti contemporaneamente, le risorse devono essere allocate a ciascuno di essi
 - ▶ Molti tipi di risorse - cicli CPU, memoria principale, file, dispositivi I/O.
- **Contabilità** - tenere traccia di quali utenti utilizzano quanto e quali tipi di risorse del computer
- **Protezione e sicurezza** - I proprietari delle informazioni archiviate in un sistema informatico multiutente o in rete devono poter controllare l'uso di tali informazioni, processi simultanei non devono interferire tra loro
 - ▶ La **protezione** deve garantire che l'accesso da parte di **processi ed utenti (interni)** alle risorse di sistema sia controllato
 - ▶ La **sicurezza rispetto ad accessi esterni** richiede l'autenticazione dell'utente e prevede la difesa dei dispositivi I/O esterni da accessi non consentiti

Panoramica dei Servizi di un SO

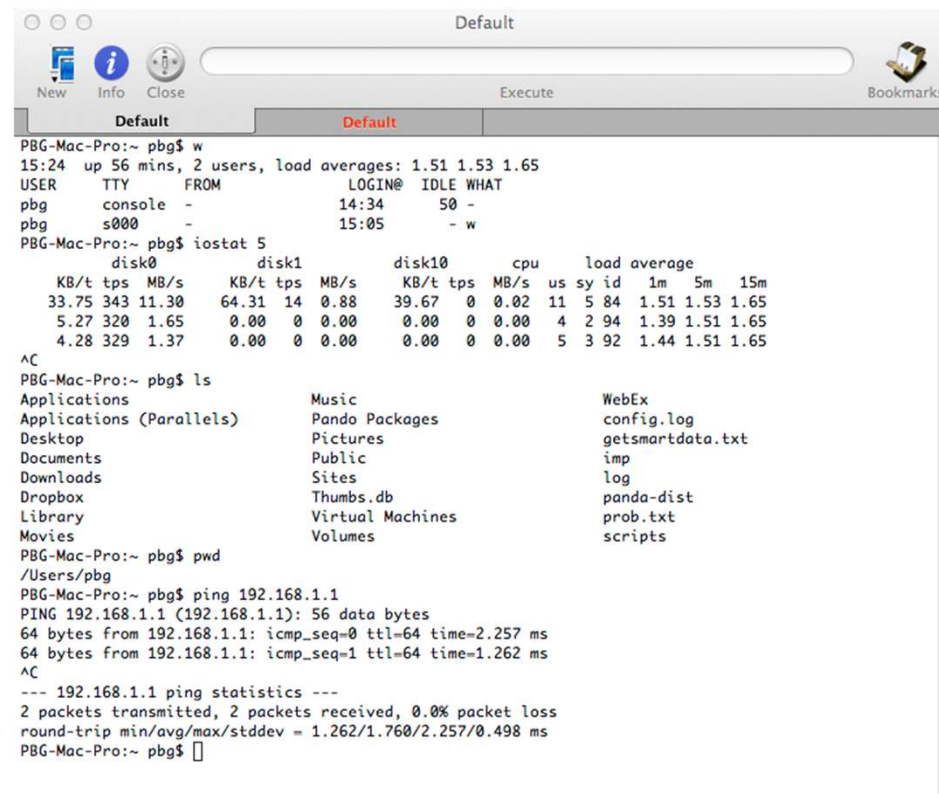
- I processi utente accedono ai servizi del Sistema Operativo mediante chiamate di sistema:
 - **Istruzioni estese** che invocano servizi gestiti dal Kernel



Interfaccia Utente - CLI

L'interprete dei comandi consente l'immissione diretta dei comandi

- A volte implementato nel kernel, a volte dal programma di sistema
- A volte più interfacce – **shells**
- Principalmente recupera un comando dall'utente e lo esegue

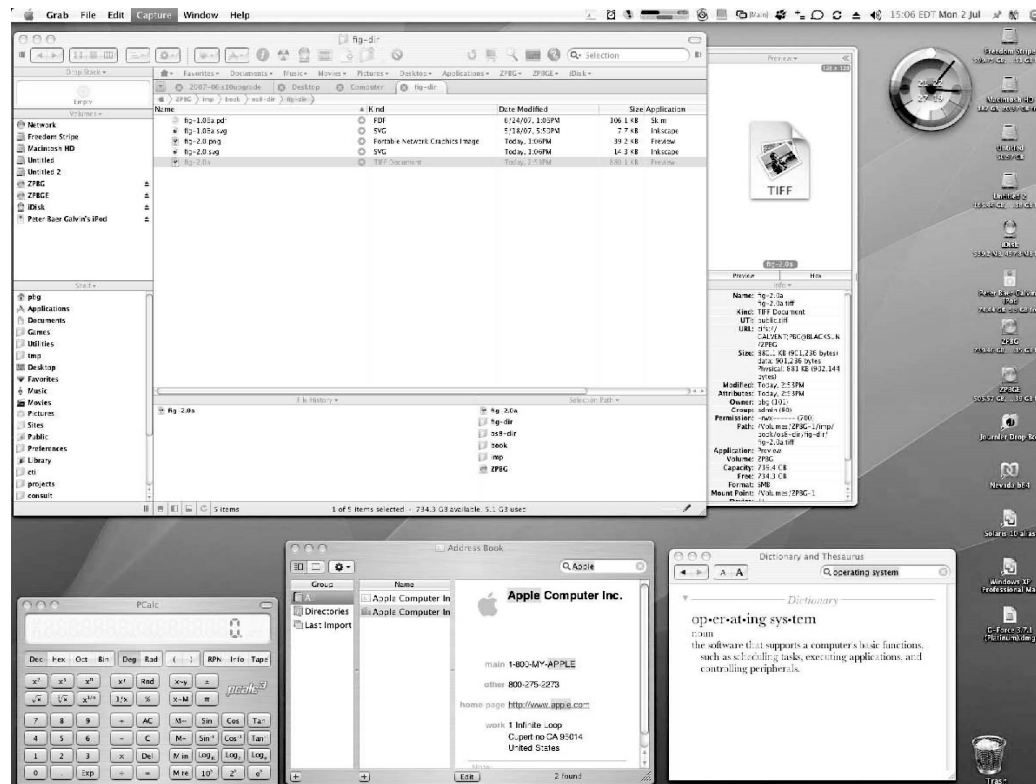


```
Default
New Info Close Execute Bookmarks
Default
PBG-Mac-Pro:~ pbg$ w
15:24 up 56 mins, 2 users, load averages: 1.51 1.53 1.65
USER      TTY      FROM          LOGIN@  IDLE WHAT
pbg       console -              14:34   50 -
pbg       s000    -              15:05   - w
PBG-Mac-Pro:~ pbg$ iostat 5
disk0      disk1      disk10      cpu      load average
KB/t tps MB/s  KB/t tps MB/s  KB/t tps MB/s  us sy id 1m 5m 15m
33.75 343 11.30 64.31 14 0.88 39.67 0 0.02 11 5 84 1.51 1.53 1.65
5.27 320 1.65 0.00 0 0.00 0.00 0 0.00 4 2 94 1.39 1.51 1.65
4.28 329 1.37 0.00 0 0.00 0.00 0 0.00 5 3 92 1.44 1.51 1.65
^C
PBG-Mac-Pro:~ pbg$ ls
Applications          Music                  WebEx
Applications (Parallels)  Pando Packages       config.log
Desktop               Pictures              getsmartdata.txt
Documents             Public                imp
Downloads             Sites                 log
Dropbox               Thumbs.db             panda-dist
Library              Virtual Machines      prob.txt
Movies               Volumes               scripts
PBG-Mac-Pro:~ pbg$ pwd
/Users/pbg
PBG-Mac-Pro:~ pbg$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1): 56 data bytes
64 bytes from 192.168.1.1: icmp_seq=0 ttl=64 time=2.257 ms
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=1.262 ms
^C
--- 192.168.1.1 ping statistics ---
2 packets transmitted, 2 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 1.262/1.760/2.257/0.498 ms
PBG-Mac-Pro:~ pbg$
```


Interfaccia Utente - GUI

Interfaccia grafica (Graphic User Interface)

- ❑ Interfaccia che usa la **metafora del desktop**
- ❑ Mouse, tastiera, schermo
- ❑ Le icone rappresentano file, programmi, azioni, ecc
- ❑ **Inventata a Xerox PARC**



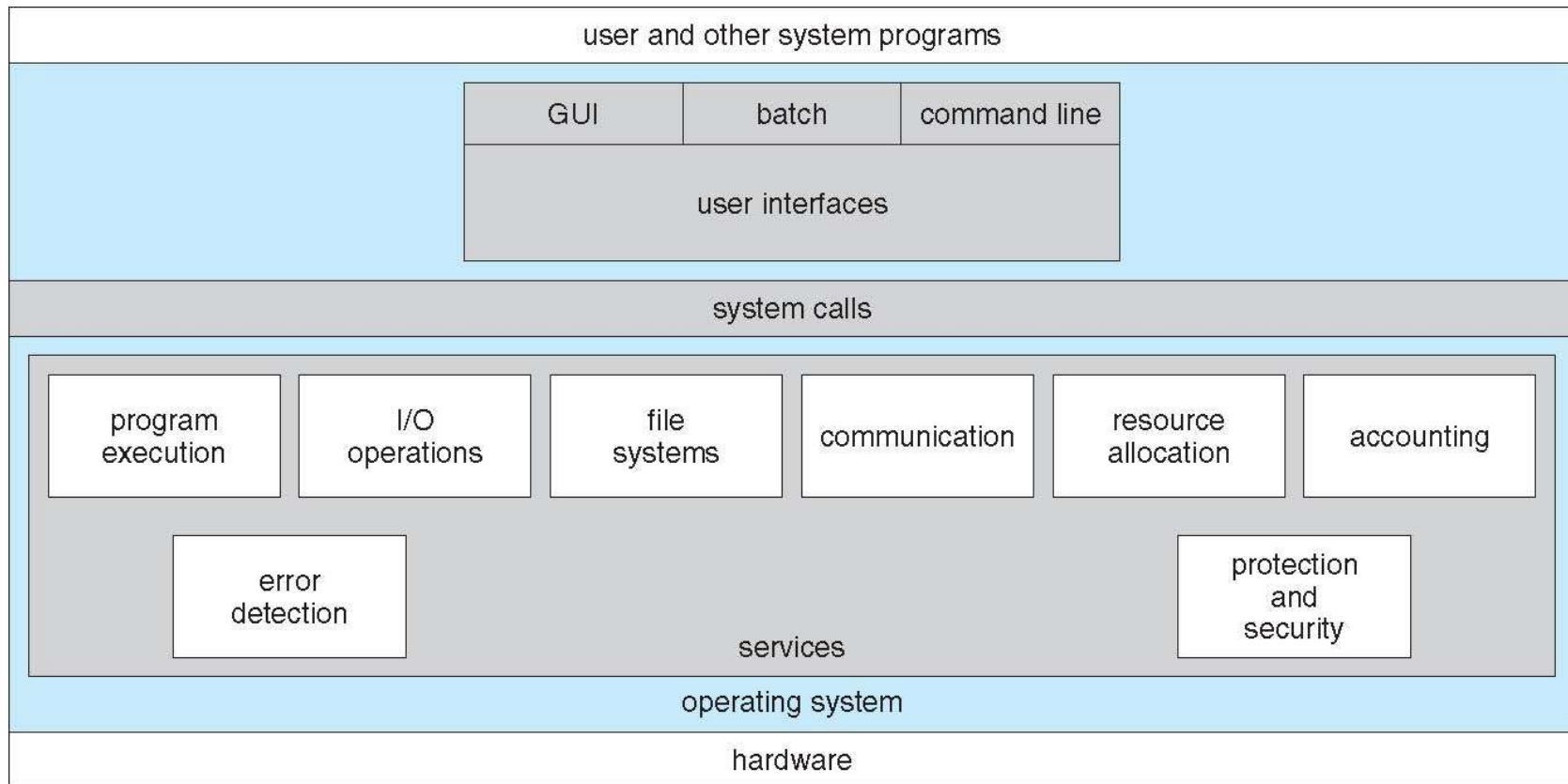
Altre Interfacce Utente

- Interfacce touch-screen
 - Senza mouse
 - Gestì per attivare azioni
- Comandi vocali
- Etc.



Panoramica dei Servizi di un SO

- I processi utente accedono ai servizi del Sistema Operativo mediante chiamate di sistema:
 - **Istruzioni estese** che invocano servizi gestiti dal Kernel



Chiamate di Sistema

- ❑ Le **chiamate di sistema** (*system call*) costituiscono un'interfaccia per i servizi resi disponibili dal sistema operativo
- ❑ Tipicamente scritte in linguaggio di alto livello (C o C++)
- ❑ Chiamate dai programmi mediante **Application Programming Interface (API)**
- ❑ Tra le API più comuni citiamo:
 - ❑ Win API per Windows (kernel32.dll, User32.dll, etc.)
 - ❑ POSIX API per POSIX-based systems (incluse tutte le versioni UNIX, Linux e Mac OS X) - **Portable Operating System Interface**
 - ❑ Java API per la Java virtual machine (JVM)

Esempio di Chiamata di Sistema

- Chiamata di Sistema per copiare il contenuto di un file in un altro file

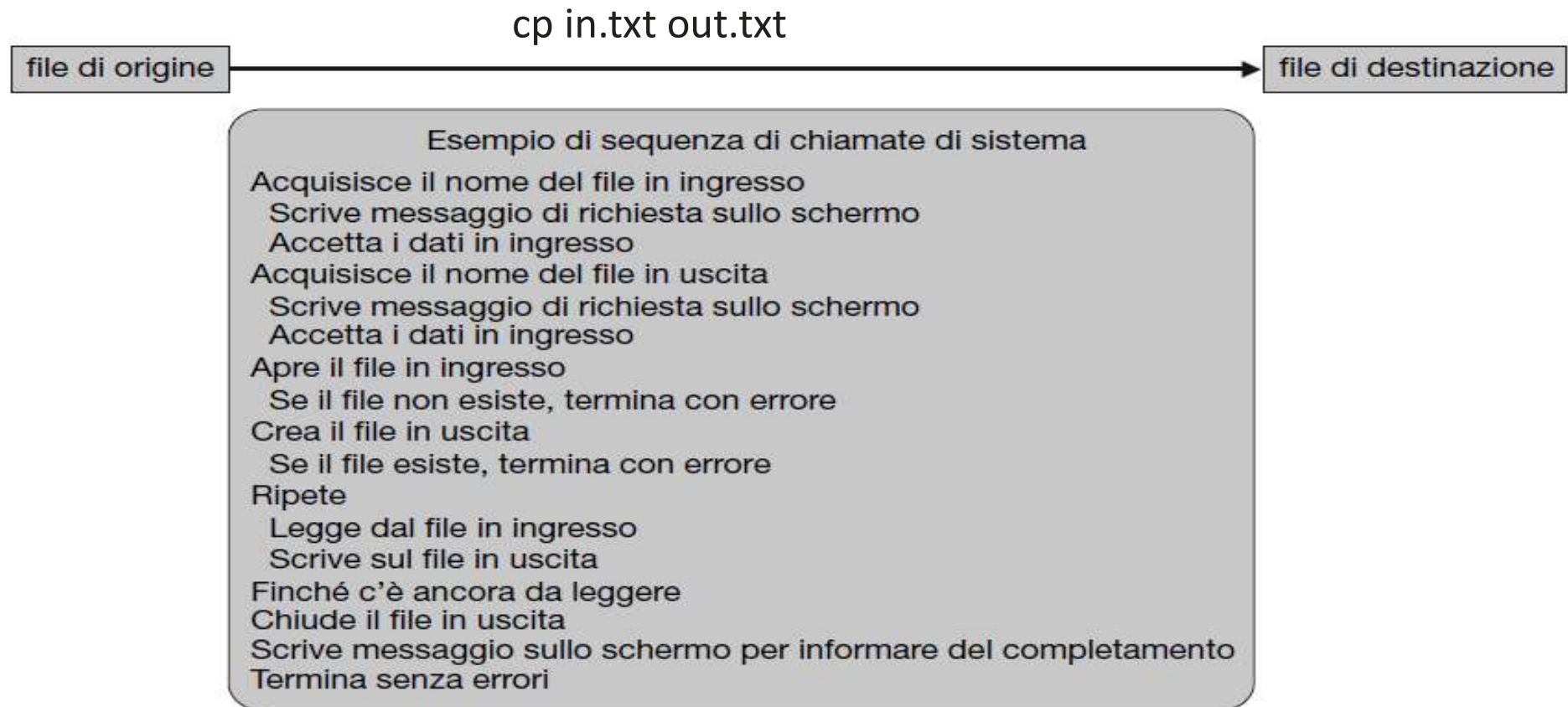


Figura 2.5 Esempio d'uso delle chiamate di sistema.

Esempio di API standard

API: Specifica un insieme di funzioni a disposizione del programmatore e dettaglia i parametri necessari per l'invocazione di queste funzioni insieme ai valori restituiti



ESEMPIO DI API STANDARD

Come esempio di API standard consideriamo la funzione `read()` disponibile in Unix e Linux. L'API per questa funzione si può ottenere digitando

```
man read
```

da riga di comando. Una descrizione di questa API è la seguente:

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count)
```

valore restituito	nome della funzione	parametri
-------------------	---------------------	-----------

Un programma che utilizza la `read()` deve includere il file `unistd.h` che, tra le altre cose, definisce i tipi di dato `ssize_t` e `size_t`. I parametri passati alla `read()` sono i seguenti:

- `int fd` — il descrittore del file da leggere
- `void *buf` — un buffer nel quale vengono messi i dati letti
- `size_t count` — il massimo numero di byte da leggere e inserire nel buffer

Quando una `read()` è completata con successo viene restituito il numero di byte letti. La `read()` restituisce 0 in caso di fine del file e -1 quando si è verificato un errore.

Relazione tra API e Chiamata

- Tipicamente un numero è associato per ogni chiamata (la **system-call interface** mantiene una tabella indicizzata)
- L'interfaccia invoca la chiamata nel kernel e restituisce lo status e valore di ritorno (dettagli nascosti gestiti da librerie di supporto)

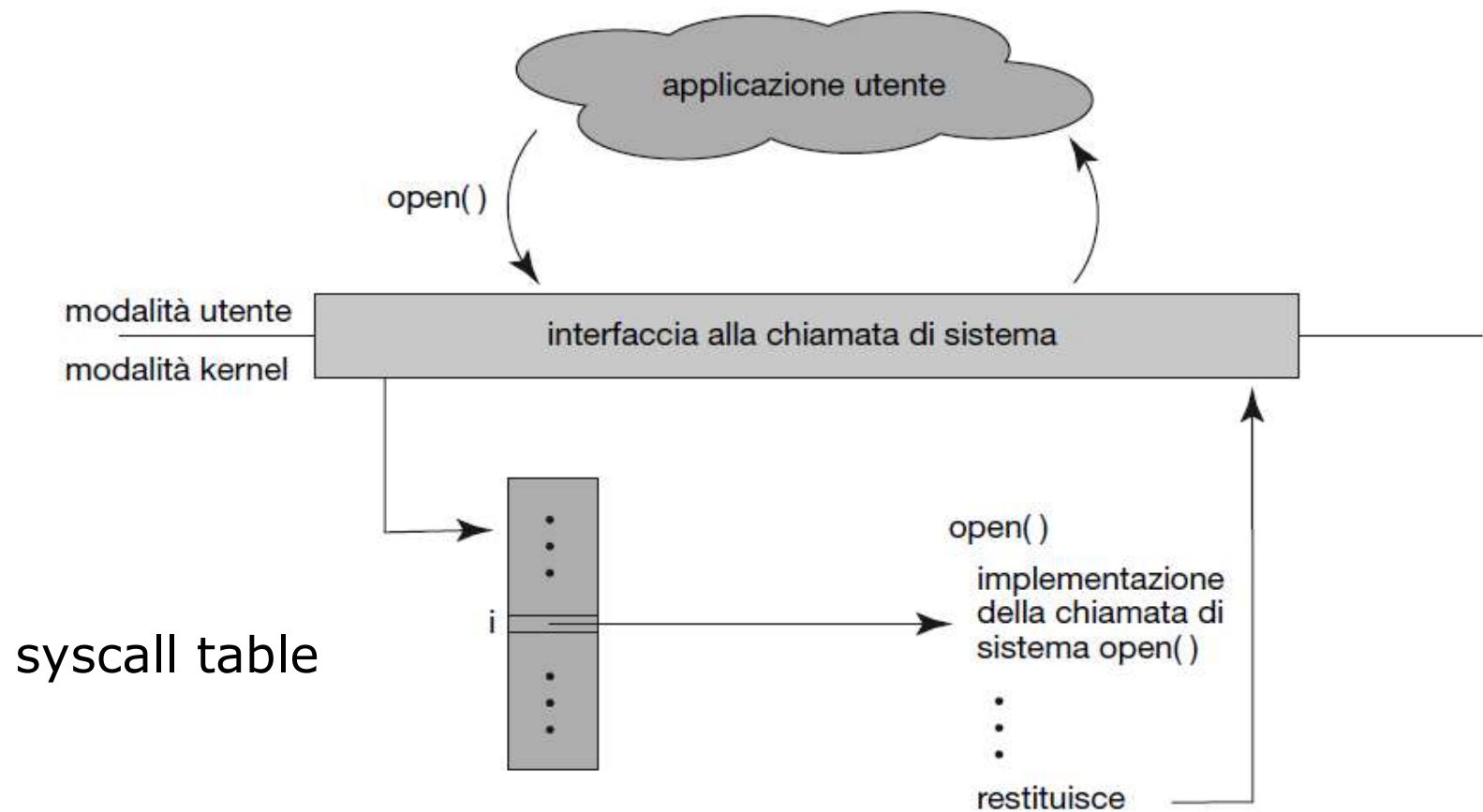


Figura 2.6 Gestione della chiamata di sistema `open()` invocata da un'applicazione utente.

Passaggio dei parametri

- Tre metodi generali utilizzati per passare i parametri al sistema operativo
 - Passa direttamente i parametri nei registri (a volte troppi parametri)
 - Parametri in un blocco, o tabella, in memoria e indirizzo del blocco passato come parametro in un registro (Linux e Solaris)
 - Parametri inseriti nello stack dal programma e estratti dallo stack dal SO

Passaggio dei parametri

- Tre metodi generali utilizzati per passare i parametri al sistema operativo
 - **Passa direttamente i parametri nei registri** (a volte troppi parametri)
 - ▶ In GNU/Linux, una chiamata di sistema accetta **al più sei parametri**
 - ▶ Es. nell'architettura Intel il registro **eax** (32 bit) o **rax** (64 bit) contiene sempre un identificatore numerico univoco della chiamata di sistema
 - ▶ Altri registri (x64)
 - * **rax** system call number/val ritorno
 - * **rdi** arg0
 - * **rsi** arg1
 - **rdx** arg2
 - **r10** arg3
 - **r8** arg4
 - **r9** arg5

Passaggio dei parametri

- Tre metodi generali utilizzati per passare i parametri al sistema operativo
 - Passa direttamente i parametri nei registri (a volte troppi parametri)
 - **Parametri in un blocco, o tabella, in memoria** e indirizzo del blocco passato come parametro in un registro (Linux e Solaris)
 - ▶ es. con `rdi = x` si può puntare al blocco

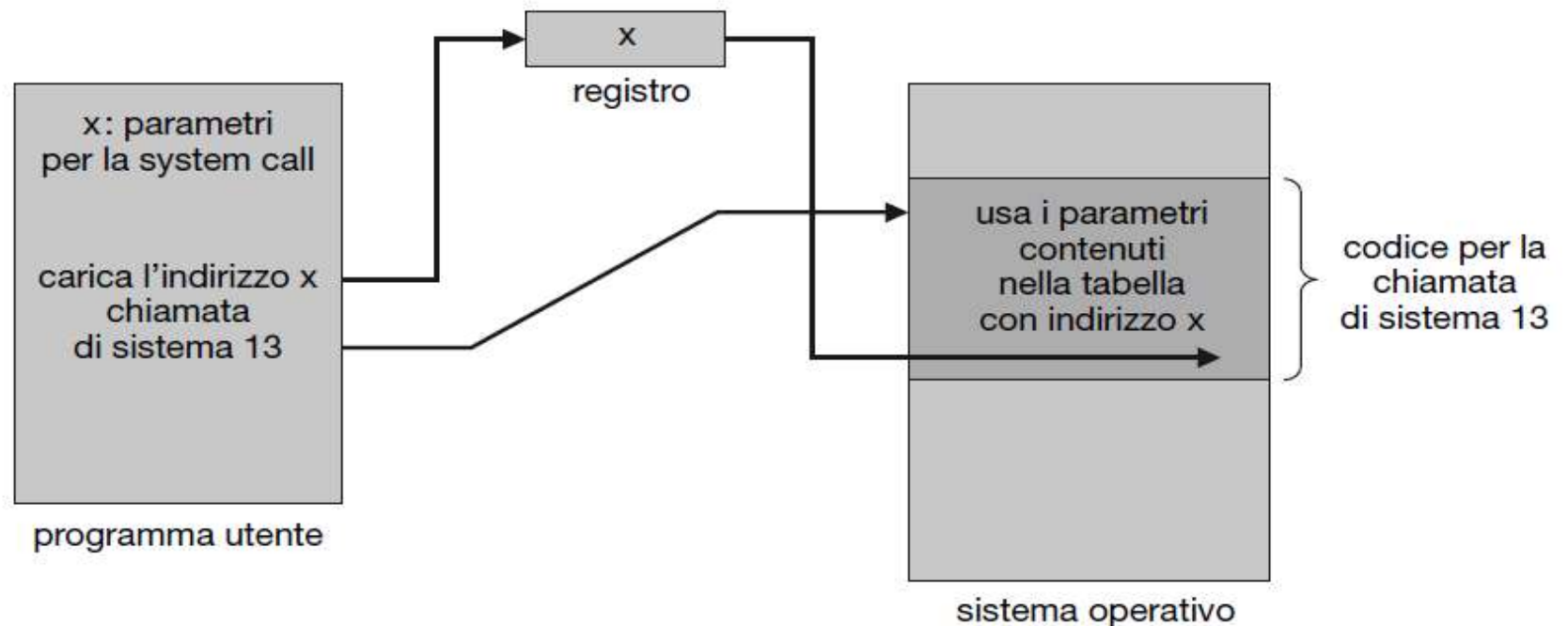


Figura 2.7 Passaggio di parametri in forma di tabella.

Chiamata di Sistema

- Parametri inseriti nello stack dal programma e estratti dallo stack dal SO
- Programma in C invoca una chiamata di sistema:
 - Invocata da una procedura con il nome della sys call *read(fd, buffer, nbytes)*
 - ▶ Restituisce numero byte o -1 nel var globale *errno*

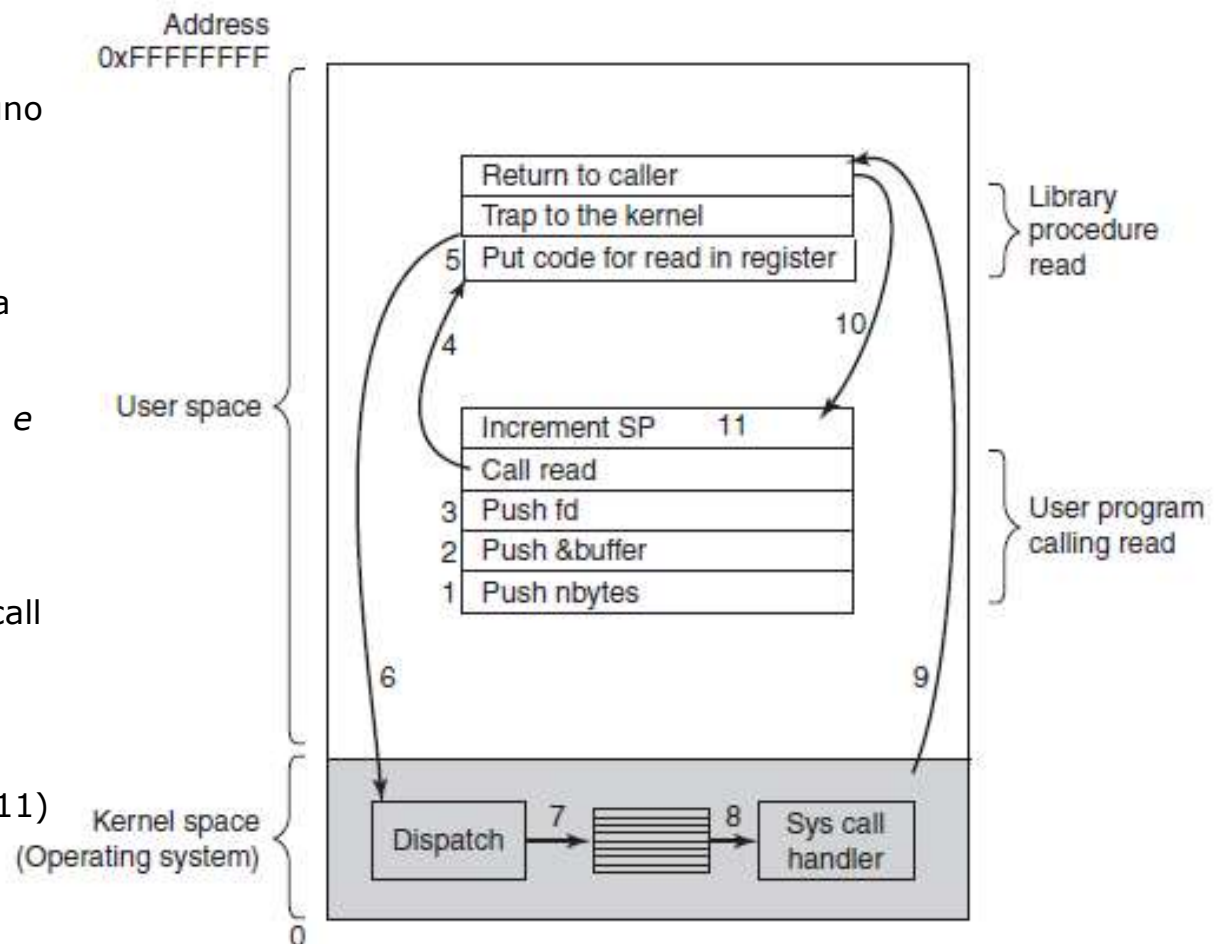
Il programma chiamante predispone la chiamata mettendo prima i paramentri in uno stack (passi 1-3) – il buffer passato per riferimento

La procedura chiama il read (passo 4) mettendo il codice della chiamata a sistema di read in un registro atteso dal Kernel

Quindi manda una istruzione *Trap* al kernel e passa il controllo al kernel in kernel mode (passi 5-6)

Il kernel trova il codice della chiamata e mediante una tabella trova l'handler della call (passi 7 e 8) finita l'esecuzione passa il risultato al chiamante (passo 9)

Il chiamante riprende il controllo in user mode (passo 10) e pulisce lo stack (passo 11) incrementando lo Stack Pointer (SP)

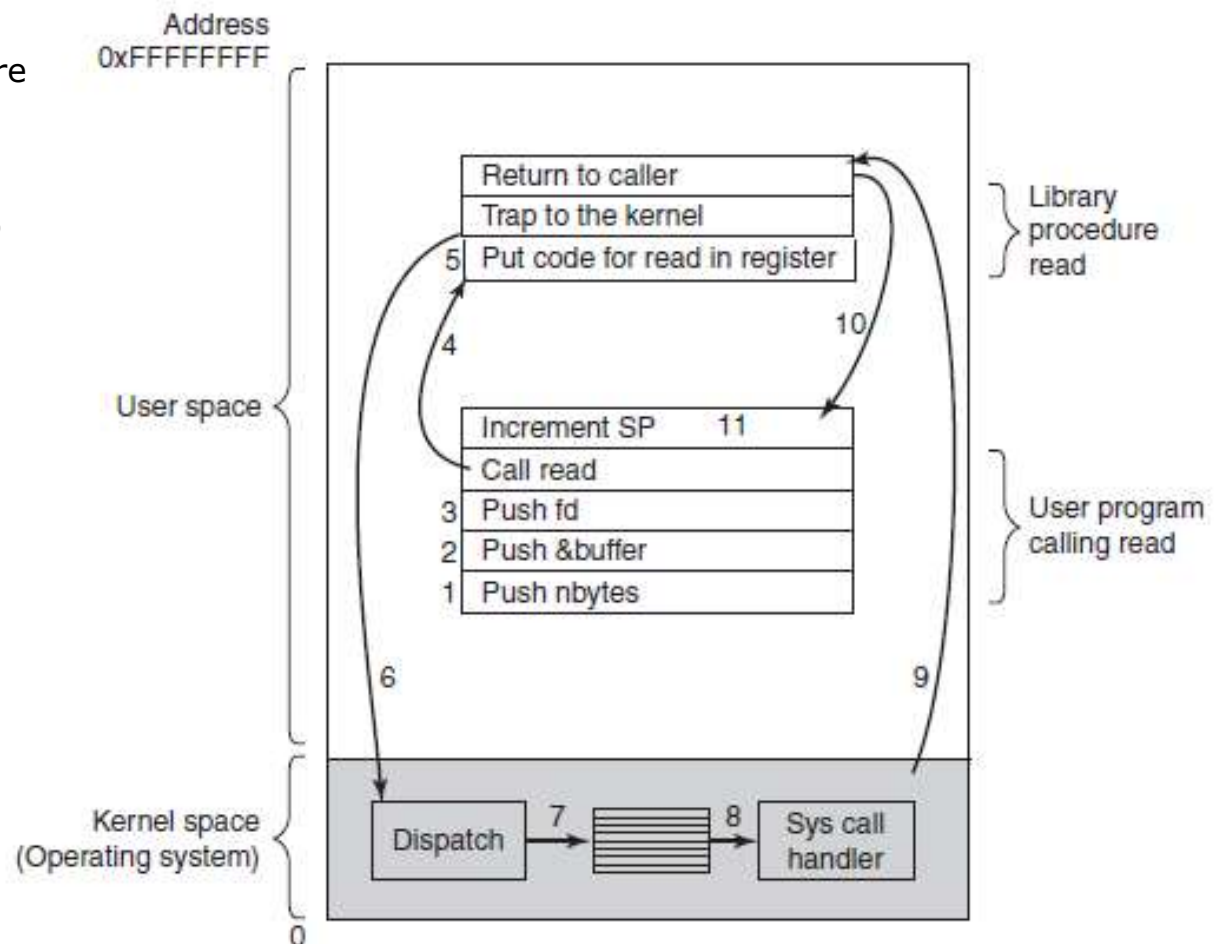


Chiamata di Sistema

- Parametri inseriti nello stack dal programma e estratti dallo stack dal SO
- Programma in C invoca una chiamata di sistema:
 - Invocata da una procedura con il nome della sys call *read(fd, buffer, nbytes)*
 - ▶ Restituisce numero byte o -1 nel var globale errno (registro rax)

Le procedure POSIX non si mappano sempre uno a uno con le system call offerte dal SO

Nella maggior parte dei casi chiamano effettivamente le sys call. Alcune eseguono operazioni senza fare il TRAP del kernel



Tipi di Chiamate di Sistema

- Controllo di processi
 - Creare e terminare processi
 - Caricare, eseguire processi
 - Ottenere e settare attributi di processo
 - Attese di tempo
 - Attese di eventi/segnali
 - Allocare memoria
 - Debugger per determinare bug
 - Meccanismi di regolazione per gestire l'accesso a dati condivisi

Tipi di Chiamate di Sistema

- Controllo di processi
 - Esempi POSIX:

```
#include <sys/types.h>  
#include <unistd.h>
```

```
pid_t getpid(void);    identificativo del processo  
pid_t getppid(void);  identificativo del genitore
```

```
pid_t fork(void);      pid_t wait(int *status);  
pid_t vfork(void);
```

```
pid_t pid;
```

```
if ( (pid=fork()) < 0 )  
    perror("fork"), exit(1);  
else if (pid != 0) {  
    // codice del padre  
} else {  
    // codice del figlio  
}
```

Tipi di Chiamate di Sistema

- Gestione di File
 - create file, delete file
 - open, close file
 - read, write, reposition
 - get and set file attributes

- Gestione di dispositivi
 - request device, release device
 - read, write
 - get device attributes, set device attributes
 - logically attach or detach devices

Tipi di Chiamate di Sistema

- Gestione informazione
 - get time or date, set time or date
 - get system data, set system data
 - get and set process, file, or device attributes

- Comunicazione
 - create, delete communication connection
 - send, receive messages if **message passing model** to **host name** or **process name**
 - ▶ da **client** a **server**
 - **Shared-memory** creazione ed accesso a regioni di memoria condivisa
 - trasferire informazione di stato
 - Installare/disinstallare dispositivi remoti

Tipi di Chiamate di Sistema

- Protezione
 - Controllo accesso a risorse
 - Gestione dei permessi
 - Gestione accesso utenti

Tipi di Chiamate di Sistema

- Controllo dei processi
 - creazione e arresto di un processo
 - caricamento, esecuzione
 - terminazione normale e anormale
 - esame e impostazione degli attributi di un processo
 - attesa per il tempo indicato
 - attesa e segnalazione di un evento
 - assegnazione e rilascio di memoria
- Gestione dei file
 - creazione e cancellazione di file
 - apertura, chiusura
 - lettura, scrittura, posizionamento
 - esame e impostazione degli attributi di un file
- Gestione dei dispositivi
 - richiesta e rilascio di un dispositivo
 - lettura, scrittura, posizionamento
 - esame e impostazione degli attributi di un dispositivo
 - inserimento logico ed esclusione logica di un dispositivo
- Gestione delle informazioni
 - esame e impostazione dell'ora e della data
 - esame e impostazione dei dati del sistema
 - esame e impostazione degli attributi dei processi, file e dispositivi
- Comunicazione
 - creazione e chiusura di una connessione
 - invio e ricezione di messaggi
 - informazioni sullo stato di un trasferimento
 - inserimento ed esclusione di dispositivi remoti
- Protezione
 - visualizzazione dei permessi di un file
 - impostazione dei permessi di un file

Figura 2.8 Tipi di chiamate di sistema.